# Chain of Responsibility Design Pattern

## Intent

Decouple the **sender** of a request from its **receiver** by giving more than one object a chance to handle it. The request travels down a **chain** of handlers; each handler either deals with it or passes it to the next one. The sender doesn't know — and doesn't care — which handler ends up doing the work.

Here the chain models a **support desk**: a request with a `Priority` is passed to Level 1 support, which either handles it or escalates to Level 2, which escalates to Level 3.

## UML class diagram

```
 <<interface>> ISupportHandler
 +handleRequest(Request)
 +setNextHandler(ISupportHandler)
       ^            ^            ^
       |            |            |
FirstLevel --> SecondLevel --> ThirdLevel   (terminal)
 BASIC?           INTERMEDIATE?   CRITICAL? else "cannot be handled"
   | no: forward     | no: forward
client -> firstLevel.handleRequest(request)  (head only)
```

---

## The players

```
request/Priority                          BASIC, INTERMEDIATE, CRITICAL
request/Request                           the thing being passed along (carries a Priority)

handler/ISupportHandler                   the handler contract: handleRequest + setNextHandler
handler/impl/FirstLevelSupportHandler     handles BASIC,        else escalates
handler/impl/SecondLevelSupportHandler    handles INTERMEDIATE, else escalates
handler/impl/ThirdLevelSupportHandler     handles CRITICAL,     else "cannot be handled" (chain end)

ChainOfResponsibilityDesignPattern        wires the chain and fires one request
```

---

## The code, line by line

### `Priority` — the request classification

```
public enum Priority {
        BASIC, INTERMEDIATE, CRITICAL
}
```

- A simple enum that ranks a request. Each handler is responsible for exactly one priority level, so this enum is effectively the routing key that decides *who* handles a request.

### `Request` — the message travelling the chain

```
public class Request {
        private Priority priority;

        public Request(Priority priority) { this.priority = priority; }

        public Priority getPriority() { return priority; }
}
```

- A small immutable-ish carrier holding the `priority`. This is the object handed from handler to handler unchanged. In a real system it would carry a payload (ticket text, user id, …); here `priority` alone is enough to demonstrate routing.

### `ISupportHandler` — the handler contract

```
public interface ISupportHandler {
        void handleRequest(Request request);
        void setNextHandler(ISupportHandler iSupportHandler);
}
```

This is the **essence of the pattern**, and it is deliberately kept to just **two** methods:

- **`handleRequest(Request)`** — "try to handle this; if you can't, pass it on." Every handler implements this.
- **`setNextHandler(ISupportHandler)`** — lets you **link** one handler to the next, forming the chain. Because the successor is typed as the interface (not a concrete class), any handler can point to any other handler — the links are interchangeable.

Keeping the interface this minimal is intentional (see *Why the interface is only two methods* below).

### `FirstLevelSupportHandler`

```
public class FirstLevelSupportHandler implements ISupportHandler {

        private ISupportHandler nextSupportHandler;

        @Override
        public void handleRequest(Request request) {
                if (request.getPriority() == Priority.BASIC) {
                        System.out.println("Level 1 Support handled the request.");
                } else if (nextSupportHandler != null) {
                        System.out.println("-----Calling next handler ie." + nextSupportHandler.getClass().getName());
                        nextSupportHandler.handleRequest(request);
                }
        }

        @Override
        public void setNextHandler(ISupportHandler nextSupportHandler) {
                this.nextSupportHandler = nextSupportHandler;
        }
}
```

- **`private ISupportHandler nextSupportHandler;`** — the link to the *next* handler in the chain. It's the interface type, so this handler has no idea what concrete class comes after it — that's the decoupling.
- **`handleRequest`** implements the classic two-branch decision:

- **Can I handle it?** `if (request.getPriority() == BASIC)` → yes, this is my level, print that Level 1 handled it. The request stops here. - **Otherwise, pass it on** — `else if (nextSupportHandler != null)` guards against being the end of the chain, prints a trace line showing who it's escalating to, then calls `nextSupportHandler.handleRequest(request)`, delegating the same request downstream.

- **`setNextHandler`** just stores the successor.

`SecondLevelSupportHandler` is identical in shape, but its "can I handle it?" test is `Priority.INTERMEDIATE`.

### `ThirdLevelSupportHandler` — the end of the chain

```
public class ThirdLevelSupportHandler implements ISupportHandler {

        @Override
        public void handleRequest(Request request) {
                if (request.getPriority() == Priority.CRITICAL) {
                        System.out.println("Level 3 Support handled the request.");
                } else {
                        System.out.println("Request cannot be handled.");
                }
        }

        @Override
        public void setNextHandler(ISupportHandler iSupportHandler) {
        }
}
```

Two things make this the **terminal** handler:

- It has **no `nextSupportHandler` field**, and its `setNextHandler(...)` body is **empty** — you cannot link anything after it. It is the last stop by construction.
- Its `else` branch prints **"Request cannot be handled."** instead of forwarding. This is the chain's fallback: if the request reaches the end and still nobody has claimed it, the chain reports failure rather than silently dropping it.

### `ChainOfResponsibilityDesignPattern` — building and firing the chain

```
FirstLevelSupportHandler  firstLevelSupportHandler  = new FirstLevelSupportHandler();
SecondLevelSupportHandler secondLevelSupportHandler = new SecondLevelSupportHandler();
ThirdLevelSupportHandler  thirdLevelSupportHandler  = new ThirdLevelSupportHandler();

firstLevelSupportHandler.setNextHandler(secondLevelSupportHandler);
secondLevelSupportHandler.setNextHandler(thirdLevelSupportHandler);

firstLevelSupportHandler.handleRequest(new Request(Priority.CRITICAL));
```

- **Create** the three handlers.
- **Link** them: `first → second → third`. This is the "chain" — a singly linked list of handlers built at runtime.
- **Fire** one request into the **head** of the chain ( `firstLevelSupportHandler` ). The sender only ever talks to the first handler; it has no idea a `CRITICAL` request will actually be resolved three hops later by Level 3.

---

## Why the design decisions

### Why Chain of Responsibility at all?

Without it, the sender would need a big conditional:

```
if (priority == BASIC)             level1.handle();
else if (priority == INTERMEDIATE) level2.handle();
else if (priority == CRITICAL)     level3.handle();
```

That couples the sender to **every** handler and every routing rule. With the chain, the sender calls **one** method on **one** handler and the routing logic lives distributed across the handlers themselves. You can reorder, insert, or remove handlers without ever touching the sender.

### Why the interface is only two methods ( `handleRequest` + `setNextHandler` )?

This is the smallest contract that still expresses the pattern: one method to *attempt/handle*, one to *link the successor*. Keeping it minimal means every handler is trivial to implement and any handler can be chained to any other. (This mirrors the deliberately-simple `SupportHandler { handleRequest; setNextHandler }` shape — no base class, no builder, no registry, just the two operations that define the pattern.)

### Why does each handler hold a reference to the *next* one (as the interface type)?

So the chain is a runtime-configurable linked list. Typing the link as `ISupportHandler` (not a concrete class) is what decouples the handlers from one another — Level 1 forwards to "some next handler," never to "the Level 2 class specifically." That means you can rewire the order or swap implementations freely.

### Why the `nextSupportHandler != null` check?

It's the guard for "I'm not the last handler." A non-terminal handler that can't handle the request forwards it; but if it happens to be the tail (no successor set), the null check stops it from calling a method on `null`. The dedicated `ThirdLevelSupportHandler` makes the tail explicit, but the null guard keeps the general handlers safe regardless of where they sit.

### Why a distinct terminal handler with an empty `setNextHandler` ?

To give the chain a **guaranteed end** and a **fallback**. `ThirdLevelSupportHandler` can't be linked further (empty setter, no field) and prints "Request cannot be handled." when nothing matches. Without a terminal fallback, an unmatched request would just fall off the end of the chain unnoticed — here it's reported.

### Why send the request only to the *first* handler?

That's the payoff of the pattern: the caller interacts with a single entry point and stays ignorant of the chain's length, order, and membership. The request propagates itself. In this demo a `CRITICAL` request enters at Level 1 and is resolved at Level 3 — and `main` never mentions Level 3 in the call that fires it.

---

## Execution flow (the demo — a `CRITICAL` request)

```
main
 │  build chain:  first → second → third
 │
 └── firstLevelSupportHandler.handleRequest( Request(CRITICAL) )
          │
          │ priority == BASIC ?  no  → escalate
          │ prints "-----Calling next handler ie...SecondLevelSupportHandler"
          ▼
      secondLevelSupportHandler.handleRequest( Request(CRITICAL) )
          │
          │ priority == INTERMEDIATE ?  no  → escalate
          │ prints "-----Calling next handler ie...ThirdLevelSupportHandler"
          ▼
      thirdLevelSupportHandler.handleRequest( Request(CRITICAL) )
          │
          │ priority == CRITICAL ?  YES
          └── prints "Level 3 Support handled the request."   ← request consumed, chain stops
```

**Console output:**

```
Chain Of Responsibility Design Pattern
-----Calling next handler ie.com.design.patterns.chainofresponsibility.handler.impl.SecondLevelSupportHandler
-----Calling next handler ie.com.design.patterns.chainofresponsibility.handler.impl.ThirdLevelSupportHandler
Level 3 Support handled the request.
```

Try other priorities to see the chain stop earlier:

- `Priority.BASIC` → handled immediately by Level 1, no escalation.
- `Priority.INTERMEDIATE` → Level 1 escalates, Level 2 handles.
- (a hypothetical unmatched priority) → falls through to Level 3's `else` → "Request cannot be handled."

---

## Notes / possible improvements (not changed in the code)

- **Each handler stops at "handled".** This is the *pure* Chain of Responsibility where exactly one handler consumes the request. A variant lets every handler act *and* forward (e.g. logging/validation pipelines) — that's the same pattern with the "stop" removed.
- **The chain is wired by hand** in `main` via `setNextHandler`. In a Spring service you'd typically inject the handlers (e.g. an ordered `List<ISupportHandler>`) and link them automatically, but hand-wiring keeps the mechanism visible here.
- Unlike some sibling modules, this one is a **plain `main`** with no `SpringApplication.run(...)` — the pattern needs no container.

---

## Relationship to the other Behavioral patterns

- **Chain of Responsibility (this module)** passes a request along a line of handlers until one handles it — the sender picks *nothing*, the chain decides.
- **Command** turns a request into an object you can queue/undo — it's about *what* to do, not *who* does it.
- **Strategy** picks *one* algorithm up front — no pass-along; the caller chooses the single handler directly.

Reach for Chain of Responsibility when **multiple objects might handle a request**, the handler set or order should be configurable, and the sender shouldn't be coupled to the specific receiver.